Software Systems

Tutorial 2

Class Diagram - Requirements Model

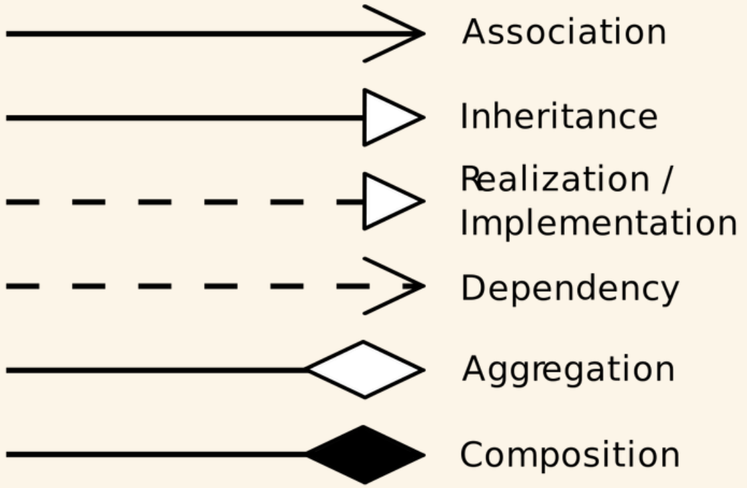
Identifying Classes

- Look for ***entities*** (or nouns) - for example: Cruise, Room, etc
- Keep classes that have ***data*** and ***behavior***.
- Remove things that are attributes of something else - for example : Price is an entity but it is a property of room or package. Since it does not have behavior and any other properties, it can be an ***attribute*** of class Room.
- Actions can be part of classes as ***methods*** *(behavior)*.

Identifying Classes

Candidate	Keep as	Reason
Price	Attribute	Each room or cruise *has a price*, but Price itself has no behavior — it's just a number or a value field inside another class (RoomPackage.price).
Payment	Class	Payment has its own data (amount, method, status) **and** behavior (process(), refund()), so it deserves its own class.
Book Cruise	Method	This is an *action* performed by a Customer or System, not a thing. So it becomes a method, e.g., Customer.bookCruise().

Class Relationships

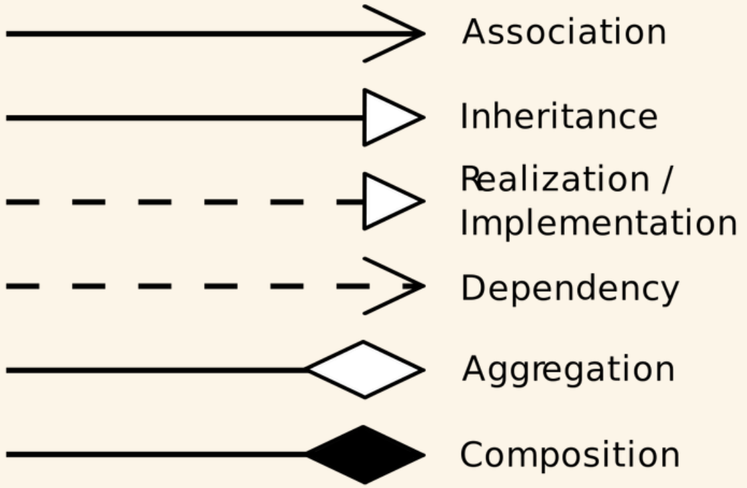


- **Association:** A basic connection showing that two classes **know** about or **use** each other.
  Example: A Customer makes a Reservation.

- **Inheritance (Generalization):** One class is a specialized version of another.
  Example: OnlinePayment inherits from Payment.

- **Realization (Interface Implementation):** A class provides the behavior promised by an interface.
  Example: CreditCardPayment implements PaymentProcessor.

Class Relationships



- **Dependency:** A temporary or “**uses**” relationship; one class depends on another to perform a task but doesn’t own it.
  Example: BookingService uses PaymentGateway to process a payment.

- **Aggregation:** “Has-a” relationship where the part can exist independently of the whole.
  Example: A Cruise has many Excursions, but an excursion can exist without that cruise.

- **Composition:** Strong “part-of” relationship where the part cannot exist without the whole.
  Example: A Reservation contains ReservationDetails; delete the reservation → details deleted too.

Association v/s Dependency

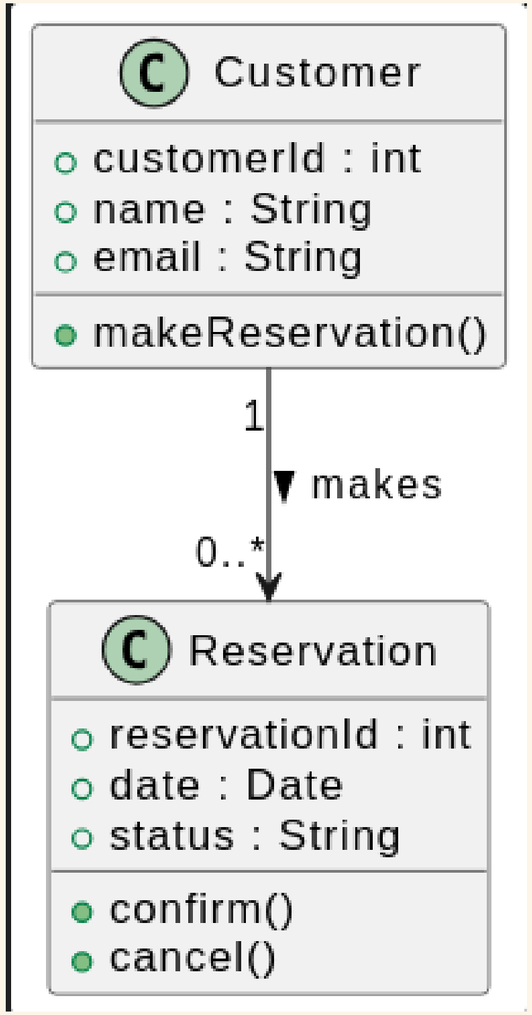
Both use another class, but association means it keeps that relationship in its state, i.e., **knows** the other class. (Long term)

Dependency means it just **uses** it for a moment. (Short term)

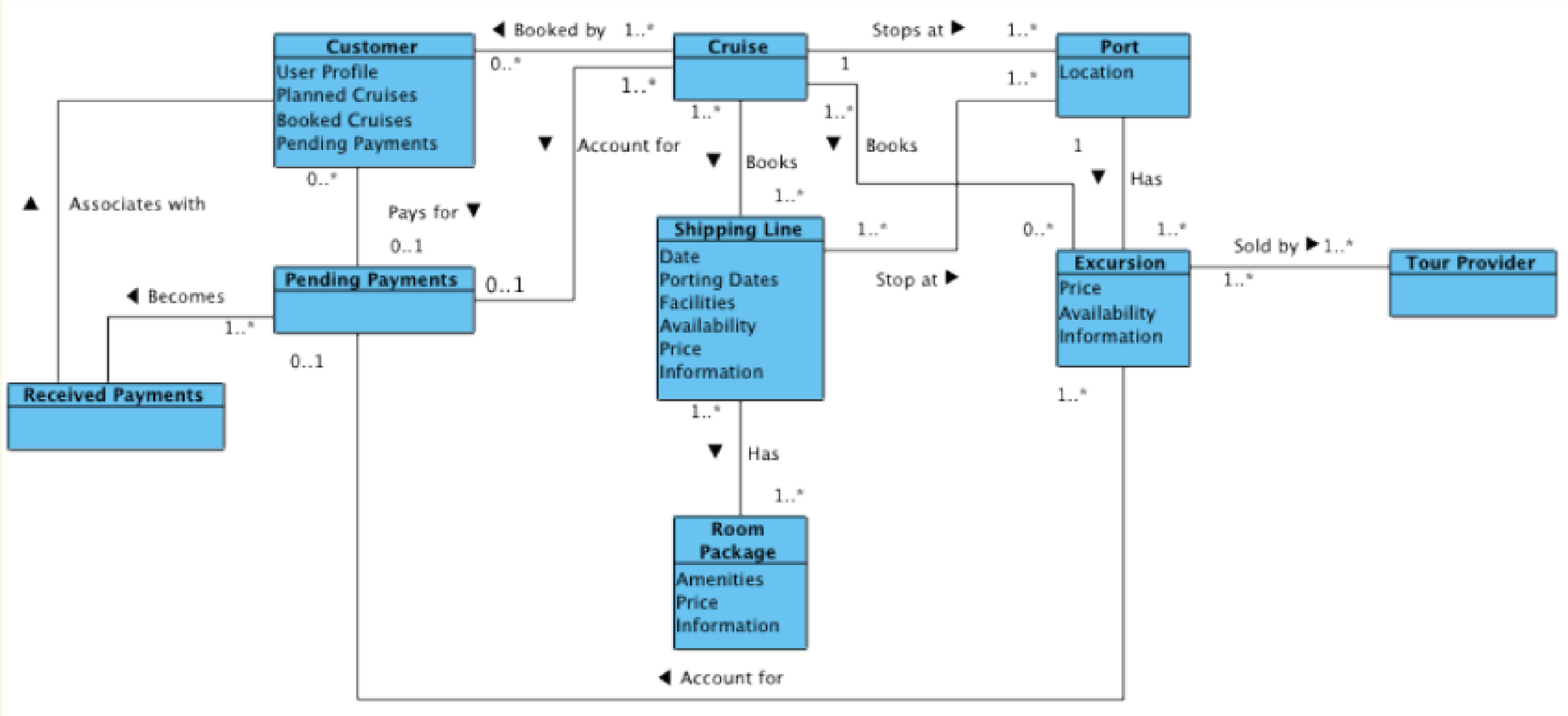
```
// Association: long-term link
class Customer {
    Reservation* reservation;   // stored reference
};

// Dependency: short-term use
class BookingService {
public:
    void processPayment(PaymentGateway gateway) {  // used temporarily
        gateway.authorize();
    }
};
```

Class Relationships

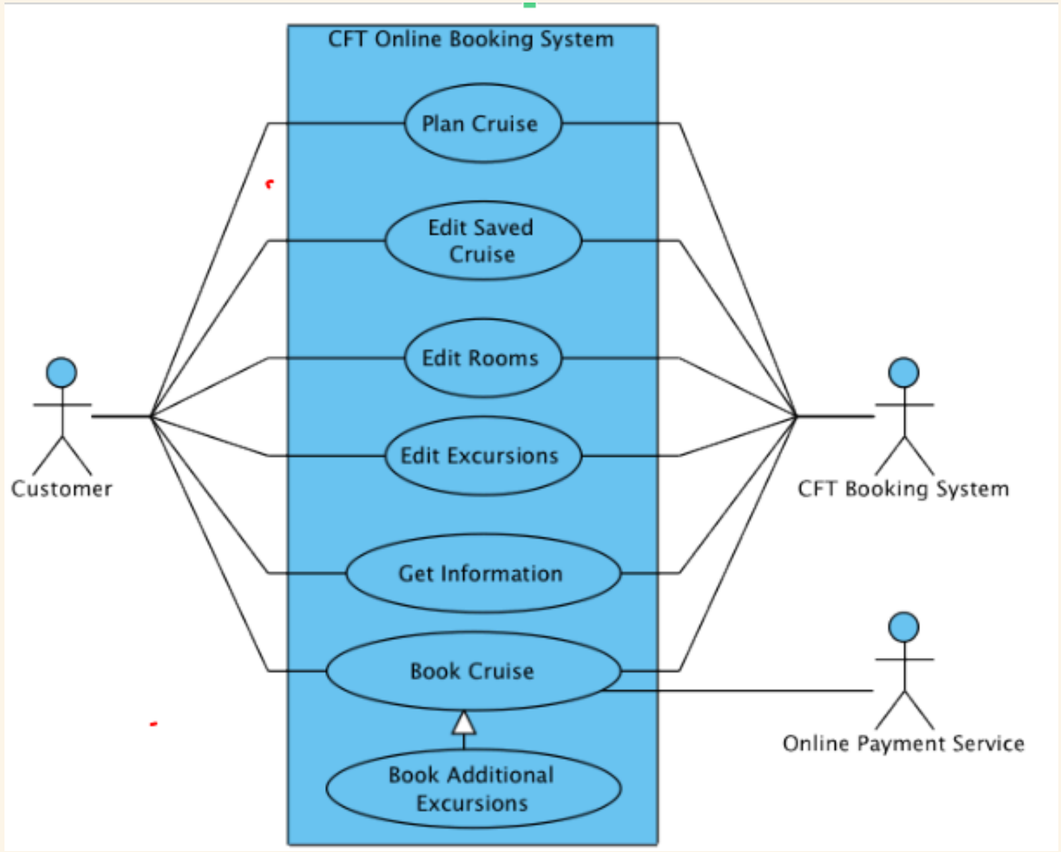
- ***Entities*** (or nouns) - Customer, Reservation
- Reservation sounds like a method, but it is an entity created as a result of the method *reserve*.
- A reservation has details like ID, customer, room, etc; methods like confirm, cancel. Hence it is a separate class.
- Relationship: **Association** (Customer uses / knows reservation)
- Multiplicity: 1 customer can make any number of reservations. 1 reservation belongs to 1 customer.



Cruise Service Model - Find Mistakes



Use Case Diagram - Find Mistakes



Detailed Use Case - Find Mistakes

Element	Description
Actors	Customer, **System**
Preconditions	Customer is logged in. Cruise availability has been verified.
Main Flow	1. Customer selects a cruise.2. System shows available rooms.3. Customer chooses a room and confirms booking.4. System **books cruise and charges card automatically**.5. System sends confirmation.
Alternate Flow	(a) If payment fails, system cancels booking and exits.
Postconditions	**TRUE**